

# Une formalisation d'ASN.1

## Soutenance de thèse de doctorat

Christian Rinderknecht

2 décembre 1998

# Plan

- ▶ Communication hétérogène entre machines distantes : langage ASN.1
- ▶ Pourquoi formaliser ASN.1 ?
- ▶ Formalisation et résultats
- ▶ Perspectives

# Communication, hétérogénéité et ASN.1

La variété des architectures et des langages de programmation rend nécessaire la définition d'un format de données universel pour communiquer.

*Abstract Syntax Notation One* (ASN.1) est un langage de *spécification* qui ne permet que la définition de types et de valeurs (pas de programmation).

Il est normalisé par l'ISO.

ASN.1 est complété par des *codages* qui transforment les valeurs ASN.1 en suites de bits :

- ▶ *Basic Encoding Rules* (BER)
- ▶ *Packed Encoding Rules* (PER)

# Fragments d'ASN.1

## ► Construction par répétition

`A ::= SET OF INTEGER`

`empty A ::= {}`

`small A ::= {7, 9, 1, 1, 3}`

## ► Construction par agrégation

`Coordinates ::= SET {x INTEGER, y INTEGER}`

`point Coordinates ::= {x 45, y 23}`

Comment le récepteur distinguera les deux champs ? Avec les *étiquettes* :

`Coordinates ::= SET {x [0] INTEGER, y [1] INTEGER}`

- Tous les champs de SET doivent avoir des étiquettes distinctes.
- Les étiquettes peuvent être calculées par le compilateur ASN.1
- L'étiquetage n'affecte pas la syntaxe des valeurs.

# Fragments d'ASN.1 (continué)

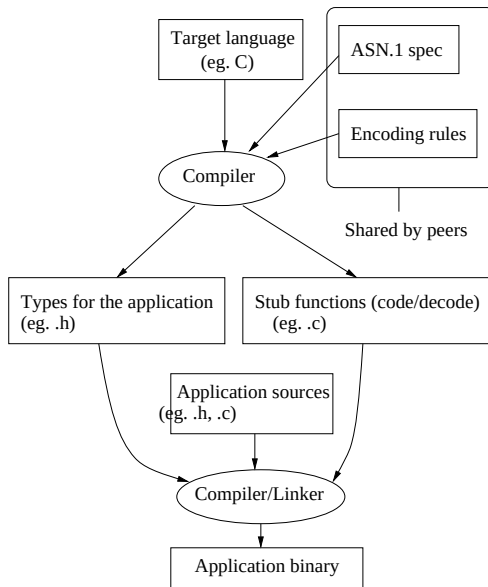
## ► Construction par union

```
T ::= CHOICE {  
    x REAL,  
    y INTEGER,  
    z BOOLEAN  
}  
v T ::= z : TRUE
```

## ► Illegal ::= CHOICE { a CHOICE {aa INTEGER, ab BOOLEAN}, b CHOICE {ba BOOLEAN, bb NULL} }

- Tous les champs de CHOICE doivent avoir des étiquettes distinctes, en parcourant en profondeur le type.

# ASN.1 et la chaîne de compilation



# Problématique

- ▶ Pas de livres sur ASN.1
- ▶ Normes volumineuses et complexes
- ▶ Normes *de plus en plus* volumineuses et complexes
- ▶ Compilateurs non conformes à la norme

# Exemples

- ▶ Une spécification incorrecte est mal rejetée :

```
T ::= T produit typedef T T;  
avant d'être rejetée par le compilateur...C !
```

- ▶ Une spécification correcte est rejetée :

```
T ::= SET (WITH COMPONENT (0)  
          | WITH COMPONENT (1)) OF INTEGER
```

- ▶ Une spécification incorrecte est acceptée :

```
a INTEGER (0<..9) ::= 0
```



# Incomplétudes de la norme

- ▶ L'abréviation de valeurs

a  $T1 ::= \langle \textit{ici une valeur de type } T1 \rangle$

b  $T2 ::= a$

Quelle condition sur  $T1$  et  $T2$  ?

- ▶ Termes récurifs

$T ::= \text{SET OF } T$

$x \ T ::= \{x\}$

$y \ T ::= \{\{\}\}$

# Pourquoi formaliser ASN.1

Attitudes possibles :

- ▶ « Cas improbables en pratique »
  - ▶ Pas forcément évident pour les spécifications normalisées (ex. la norme X.500 – l'annuaire – présente des types récurifs).
  - ▶ Risqué pour les spécifications privées.
- ▶ « Un bon outil est fondé sur une bonne sémantique »

Description formelle des structures du langage.

# Avantages de la formalisation

1. Pour les comités de normalisation
  - 1.1 Recherche d'incomplétudes, d'ambigüités, d'incohérences et de restrictions inutiles.
  - 1.2 Si un codage est formalisé aussi, des preuves de propriétés (ex. correction) sont envisageables.
2. Pour les spécifieurs

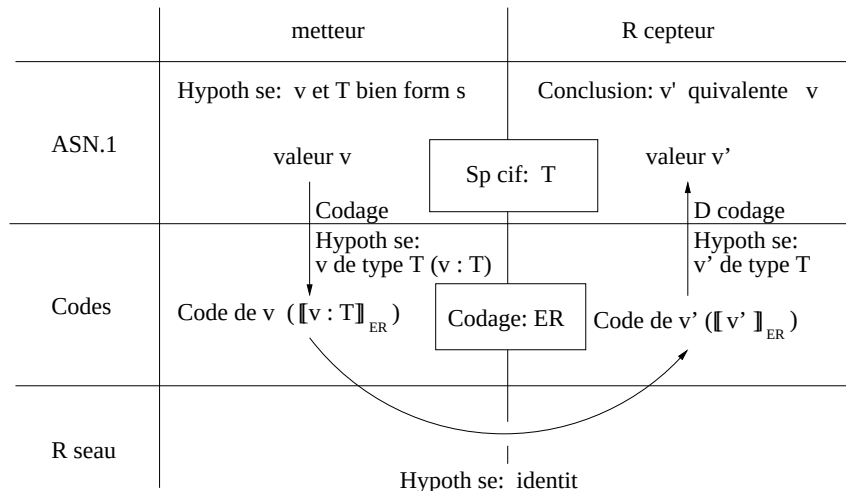
La formalisation peut aussi définir les algorithmes de la phase d'analyse d'un compilateur ASN.1.

# La formalisation

Simplifications et hypothèses :

- ▶ Nous occultons la couche applicative (et sa syntaxe concrète, ie. son langage de programmation). Ainsi nous supposons que
  - ▶ les valeurs sont en ASN.1 avant d'être codées.
  - ▶ le décodage se fait vers ASN.1.
- ▶ Nous définissons des codes de moins bas-niveau que les bits.

# Objectif



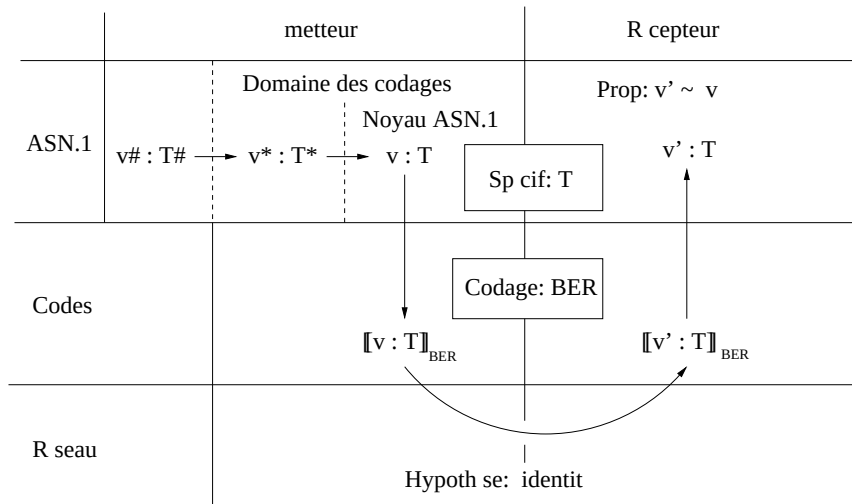
# Difficultés

1. Les codages normalisés sont des applications partielles (ie. s'appliquent à un sous-ensemble d'ASN.1).
2. Les nombreuses constructions ASN.1 impliquent que
  - ▶ les relations logiques ont un très grand nombre de règles.
  - ▶ l'équivalence de valeurs n'est pas intuitive.

# Nouvelle donne

1. Nous réécrivons ASN.1 vers le domaine des BER et définissons des classes d'équivalence de codes.
2. Nous réécrivons encore la spécification ASN.1 vers un noyau, pour rendre les preuves plus simples (environnements canoniques).
3. L'équivalence des valeurs dans ce noyau est l'identité modulo l'ordre de certains sous-termes (types SET et SET OF).

# Nouvel objectif



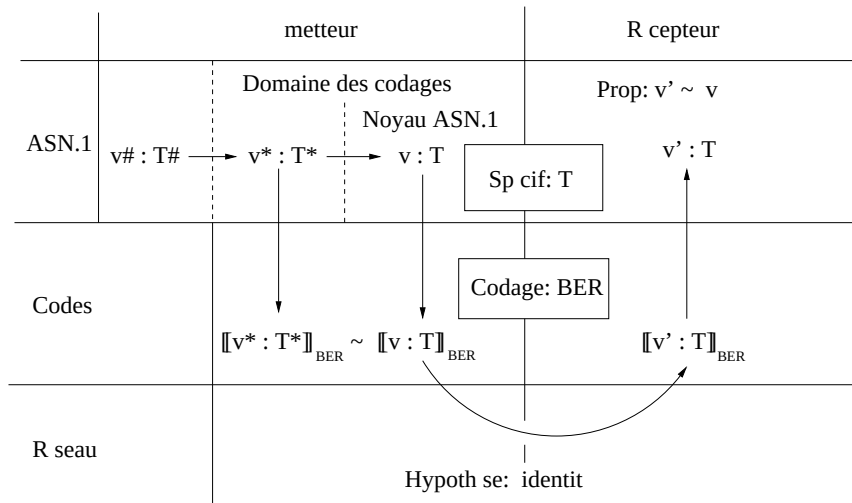


## Remarques

- ▶ Les réécritures vérifient la bonne formation des constructions ASN.1 en les simplifiant.
- ▶  $v^*$  possède un code, donc il faut prouver en sus :

$$\llbracket v^* : T^* \rrbracket_{\text{BER}} \sim \llbracket v : T \rrbracket_{\text{BER}}$$

# Nouvel objectif équivalent



# Précisément

Formalisme : règles d'inférence.

Définissons

- ▶ la spécification  $\Gamma$  dans le noyau d'ASN.1.
- ▶ une valeur  $v$  et un type  $T$  dans  $\Gamma$ .
- ▶ un contrôle des types, noté  $\Gamma \vdash v : T$
- ▶ le codage BER, noté  $\Gamma \vdash v : T \Rightarrow \bar{v}$
- ▶ le décodage BER, noté  $\Gamma \models \bar{v} : T \rightarrow v'$
- ▶ l'équivalence de valeurs, notée  $\Gamma, T \vdash v \sim v'$

## Précisément (continué)

Nous voulons alors prouver la *correction du codage BER* :

|        | <b>Théorème</b>                             | <b>Taille</b>                          |
|--------|---------------------------------------------|----------------------------------------|
| Soient | $v, T \in \Gamma$                           | 199 règles (réécriture vers $\Gamma$ ) |
| Si     | $\Gamma \vdash v : T$                       | 18 règles                              |
| et     | $\Gamma \vdash v : T \Rightarrow \bar{v}$   | 21 règles (BER)                        |
| alors  | $\Gamma \models \bar{v} : T \rightarrow v'$ | 21 règles                              |
| et     | $\Gamma, T \vdash v \sim v'$                | 5 règles                               |

Taille de la preuve : 70 pages du manuscrit.

# Remarques

- ▶ Les BER plongent toute l'information de contrôle (ie. les types des valeurs) dans les codes, mais ignorent les contraintes de sous-typage.
  - ▶ Avantage : Intuitif.
  - ▶ Inconvénient : Le ratio «contenu utile/contrôle» est faible.
- ▶ La preuve est en fait divisée en quatre parties :
  1. Unicité du contrôle (syntaxique) des types
  2. Unicité du codage
  3. Unicité du contrôle sémantique des types ( $\Rightarrow$  décodage)
  4. Correction proprement dite

# Unicité du contrôle des types et du codage

$T ::= \text{CHOICE } \{a \text{ REAL}, a \text{ BOOLEAN}\}$

$x \ T ::= a : 0$

Pour contrôler le type de  $x$  il faut peut-être rebrousser chemin (non-déterminisme). Le codage peut être incohérent :

$$\llbracket x : T \rrbracket_{\text{BER}} = \llbracket 0 : \text{REAL} \rrbracket_{\text{BER}} \text{ ou } \llbracket 0 : \text{BOOLEAN} \rrbracket_{\text{BER}}$$

Solution : *Contraindre les labels pour impliquer toujours l'unicité* (suffisant) ou fusionner contrôle des types et codage.

# Unicité du décodage

```
T ::= CHOICE {a [5] INTEGER, b [5] INTEGER}  
x T ::= a : 0
```

Le décodage peut être incohérent car il n'est pas une fonction :

$$\llbracket x : T \rrbracket_{\text{BER}} = \llbracket (a : 0) : T \rrbracket_{\text{BER}} = \llbracket (b : 0) : T \rrbracket_{\text{BER}}$$

Solution : *Contraindre les étiquettes pour impliquer toujours l'unicité* (suffisant).

# Résultats

Nous avons

- ▶ formalisé tout ASN.1 (norme X.680) ainsi que les BER.
- ▶ découvert des incomplétudes, des restrictions inutiles et des maladresses coûteuses dans la conception d'ASN.1.
- ▶ participé aux comités ISO sur ASN.1 pour corriger et faire évoluer la norme.
- ▶ prouvé la correction des BER.
- ▶ réalisé un analyseur de spécifications ASN.1 qui a été employé dans l'industrie (Southwestern Bell, Standard&Security, CNET) : Asno.



# Asno : Un nouvel outil

- ▶ Asno est fondé sur la formalisation.
- ▶ Écrit en Objective Caml (langage fonctionnel).
- ▶ Grammaire normalisée d'ASN.1 en YACC :
  - ~ 1000 conflits décaler/réduire.
  - ~ 100 conflits réduire/réduire.

Réécriture vers une forme LL(1) et preuve de l'équivalence des langages reconnus.

L'analyseur syntaxique reconnaît exactement la grammaire normalisée (extensions dynamiques de syntaxe incluses).

- ▶ Asno est diffusé gratuitement par l'INRIA.

# Utilisations d'Asno

- ▶ South-Western Bell (compagnie de téléphone états-unienne) :
  - ▶ Évaluation très positive (conformité de l'outil).
  - ▶ Intégration dans des outils maison (gestion de bases de données de protocoles).
- ▶ Standard & Security (compagnie privée britannique) :
  - ▶ Aide à la spécification de protocoles (avant la compilation) : messages d'erreur détaillés et finesse d'analyse.
- ▶ CNET (France Télécom) :
  - ▶ Réutilisation pour le développement d'outils.
- ▶ Université de Barcelone
  - ▶ Apprentissage d'ASN.1 et des macros SNMP (gestion de réseaux).

# Perspectives

- ▶ Formaliser la traduction d'ASN.1 vers les types d'un langage de programmation dont la sémantique est connue (pour dériver une partie de la phase finale d'un compilateur ASN.1).
- ▶ Reprendre le travail avec une formalisation des PER.
- ▶ Nous avons aussi simplifié, par réécritures, les contraintes de sous-typage pour les valider. Les BER ignorent ces contraintes mais pas les PER qui s'en servent pour la minimisation des codes : il s'agirait de poursuivre les réécritures pour que les PER soient optimales en ce sens.